

LCP: Longest Common Prefix

What "prefix" means, precisely

A **prefix** of a string is a piece that **starts at the very beginning (index 0)** and goes forward some number of characters. It's not "any matching chunk anywhere inside the string" — it specifically has to anchor at the start.

So for `"dogcar"`, valid prefixes are: `"d"`, `"do"`, `"dog"`, `"dogc"`, `"dogca"`, `"dogcar"` — that's it. `"car"` is **not** a prefix of `"dogcar"`, even though `"car"` does appear inside it — it appears at the *end*, not the *start*.

Applying that to your example

`strs = ["dogcar", "racecar", "car"]`

- Is `"car"` a prefix of `"dogcar"`? No — `"dogcar"` starts with `d`, not `c`.
- Is `"car"` a prefix of `"racecar"`? No — `"racecar"` starts with `r`, not `c`.
- Is `"car"` a prefix of `"car"`? Yes (trivially, itself).

Since `"car"` fails the prefix test on 2 of the 3 strings, it's not a valid answer under the problem's actual definition — even though it *is* a common **substring** (appears somewhere in all three).

The correct answer for `["dogcar","racecar","car"]` is `""` — because the very first characters (`d`, `r`, `c`) don't even match each other, so there's no common prefix at all, not even one character.

Why the problem specifically wants prefix, not substring — and why that changes the algorithm entirely

This distinction matters a lot for *how* you solve it, not just terminology:

Prefix matching is anchored — you're always comparing characters starting from position 0 across all strings simultaneously. This makes it a simple, fast left-to-

right scan: compare column 0 across all strings, then column 1, etc., stopping at the first mismatch.

Common substring matching (which is what your `"car"` example is really asking about) is a much harder problem — the matching chunk could start at *any* position in *any* string, and could be different starting positions in different strings. This is the "Longest Common Substring" problem, and it typically needs dynamic programming (`O(n*m)` comparing pairs of strings, or suffix trees/arrays for multiple strings) — dramatically more complex than prefix matching.

Why "always check from the front" makes sense once you see this

Since a prefix is *defined* as starting at index 0, there's no ambiguity about "where do I start comparing" — you always start at index 0 for every string, by definition. That's precisely what makes prefix-matching simple: you're never searching for *where* the common part begins, only *how long* it extends before breaking. That "where" question is exactly what makes substring-matching much harder — and exactly why this specific problem (asking for prefix) is tractable with a simple pass, while your "center matching" idea would actually be solving a different, harder problem.
